# Trouve ton artisan

## Dossier de projet

## pixelle2611

Création d’un site web avec React.js

23 octobre 2026

## Présentation

Un annuaire destiné aux habitants de la région Auvergne-Rhône-Alpes pour découvrir des artisans, consulter leur spécialité et leur localisation, puis les contacter depuis leur fiche.

Site : trouve-artisan.alwaysdata.net

Dépôt GitHub : pixelle2611-design/trouve-ton-artisan

## Technologies

React, React Router, Bootstrap et Sass pour l’interface. Node.js, Express et Sequelize pour le serveur. MySQL ou MariaDB pour la base de données. Nodemailer pour les messages de contact.

## Sommaire

1. Contexte et parcours utilisateur - page 2

2. Architecture de l’application - page 3

3. Base de données : MCD et MLD - page 4

4. Sécurité et accessibilité - page 5

5. Vérifications et conditions de validation - page 6

6. Veille de sécurité - page 7

7. Maquettes et prototype - page 8

8. Captures des maquettes - pages 9 et 10

9. Installation et mise en ligne - page 11

# 1. Contexte et parcours

## Objectif du site

Le projet répond au besoin de mise en relation entre les habitants et les professionnels de l'artisanat. L'annuaire présente quatre catégories : Bâtiment, Services, Fabrication et Alimentation. Le jeu de données comporte 17 artisans et 15 spécialités.

## Fonctionnalités

Accueil : présentation du service, quatre étapes pour trouver un artisan et trois artisans du mois.

Consultation par catégorie : liste des professionnels avec nom, spécialité, note et ville.

Recherche : filtrage des artisans par leur nom.

Fiche : présentation, localisation, note, lien web facultatif et formulaire de contact.

Contact : nom, email, objet et message transmis au serveur. Le délai annoncé de réponse de 48 heures dépend du professionnel.

Pages complémentaires : mentions légales, données personnelles, accessibilité et cookies avec la mention « Page en construction ». Une page 404 propose un retour à l'accueil.

## Présentation responsive

Le site utilise une grille Bootstrap et des styles Sass. Sous 768 pixels, un bouton permet de déplier les catégories. Les étapes et les cartes s'adaptent à la largeur disponible ; la fiche empile la présentation et le formulaire avant le seuil ordinateur.

# 2. Architecture

## Interface React

src/main.jsx monte l'application dans le navigateur. src/App.jsx définit les routes et le gabarit commun : en-tête, contenu principal et pied de page. Les pages sont regroupées dans src/pages ; les cartes, notes, formulaires et éléments partagés dans src/components.

src/services/api.js centralise les appels fetch. Le hook useData suit le chargement et les erreurs, et ignore une réponse arrivée après le changement de page. React Router permet d'ouvrir directement une liste ou une fiche par son adresse.

## Serveur Express

server/index.js initialise la connexion à la base et les serveurs, puis gère leur arrêt. Les routes métier sont regroupées dans server/routes ; le transport SMTP dans server/services ; les protections, la passerelle et la gestion des erreurs dans server/middleware. Les modèles et la validation possèdent leurs propres modules.

## Chemin d'une requête

Le navigateur appelle /api. Une passerelle vérifie la méthode et la route avant de transmettre la demande à une API interne sur 127.0.0.1. Une clé aléatoire générée au démarrage est réservée au serveur. Sequelize lit les données, assemble les relations et renvoie uniquement les attributs publics.

Pour un contact, le serveur contrôle les champs, retrouve l'artisan en base et prépare le courriel. Le destinataire provient de la base ; l'adresse du visiteur figure dans Reply-To. Sans configuration SMTP, le serveur renvoie une indisponibilité et n'affiche pas de succès fictif.

## Routes principales

GET /api/categories : catégories.

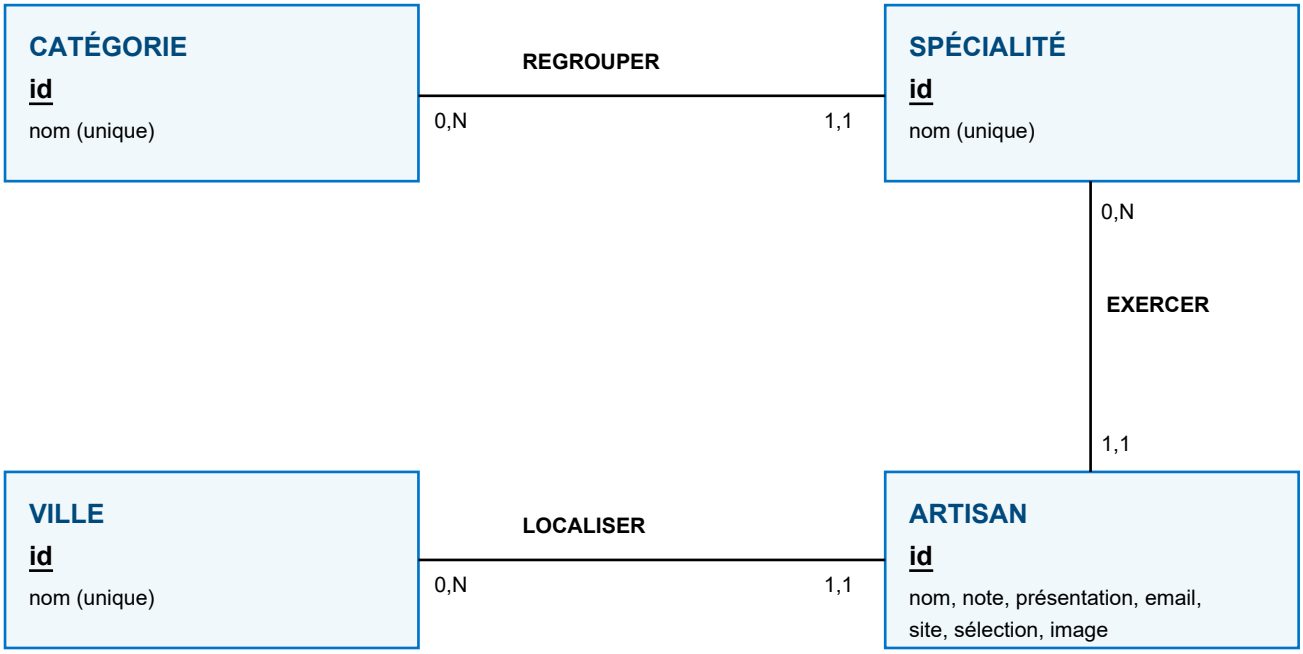
GET /api/artisans : liste, recherche et sélection du mois.

GET /api/artisans/:id : fiche publique.

POST /api/artisans/:id/contact : validation et envoi du message.

# 3. Base de données

## MCD : entités et cardinalités




## MLD : tables relationnelles

Convention : clé primaire **grasse et soulignée** ; clé étrangère préfixée par # ; colonne UNIQUE **grasse**. Le # sert à la notation et ne figure pas dans les noms SQL.

categories (**id** INT, **name** VARCHAR(60)).

specialties (**id** INT, **name** VARCHAR(80), #category_id INT).

cities (**id** INT, **name** VARCHAR(100)).

artisans (**id** INT, name VARCHAR(120), rating DECIMAL(2,1), #city_id INT, about TEXT, email VARCHAR(254), website VARCHAR(255), top BOOLEAN, image VARCHAR(255), #specialty_id INT).

#category_id vers categories.id ; #city_id vers cities.id ; #specialty_id vers specialties.id. Tous les champs sont obligatoires sauf website. Identifiants auto-incrémentés ; top vaut FALSE ; image vaut /artisan.svg ; note entre 0 et 5.

La relation Ville élimine les répétitions de localité dans artisans. Le jeu fourni comporte 14 villes et 17 artisans. L'unicité du nom est adaptée au jeu régional fourni ; un annuaire étendu avec communes homonymes doit introduire un code INSEE. Les cardinalités et dépendances retenues respectent les trois premières formes normales dans ce périmètre.

# 4. Sécurité et accessibilité

## Validation des entrées

Le serveur contrôle les champs obligatoires, types, longueurs, identifiants et format email. Les valeurs utilisées dans les en-têtes email refusent les retours de ligne. Le JSON est limité à 16 Ko. Les paramètres multiples inattendus et les caractères joker de recherche sont rejetés. Les requêtes sont construites avec Sequelize.

## Protection des accès

Helmet fournit les en-têtes de sécurité. La passerelle expose uniquement les routes de consultation et de contact. Les limites sont de 100 requêtes API par minute et cinq tentatives de contact par période de 15 minutes. Le POST exige l'origine configurée ; cette vérification limite les soumissions depuis un autre site sans constituer une authentification.

Les secrets SQL et SMTP sont lus depuis l'environnement et exclus du livrable. Le compte SQL de l'application est limité à la lecture sur la base de déploiement. Les emails des artisans ne figurent pas dans les réponses publiques. Les messages sont envoyés en texte ; les accès aux fichiers et URL sont désactivés dans le transport SMTP. TLS est exigé en production.

## Gestion des erreurs

Les erreurs sont centralisées et renvoyées sous forme de messages génériques, sans pile d'exécution ni identifiants de connexion. L'interface affiche les états de chargement, les résultats vides et les erreurs. La passerelle limite son attente à 15 secondes.

## Navigation accessible

Les champs possèdent des labels associés. Les navigations sont nommées, le focus est visible et un lien d'évitement mène au contenu. La note dispose d'un texte lisible ; les étoiles décoratives sont masquées aux technologies d'assistance. Le menu mobile expose aria-expanded et aria-controls, se ferme à la navigation et avec Échap ; Échap rend le focus au bouton.

Les pages disposent d'un titre et d'une description. Les images possèdent un texte alternatif, les états sont annoncés et les préférences de mouvement réduit sont prises en compte. Une revue complète au lecteur d'écran reste nécessaire avant de revendiquer une conformité WCAG.

# 5. Vérifications

## Résultats disponibles

**Tests automatisés :** 15 tests Node réussis, sans échec. Ils couvrent les champs, les identifiants, les réponses publiques, les erreurs HTTP, l'origine, les limites JSON, une panne SQL simulée, l'absence de SMTP et la limitation des contacts.

**Compilation :** build Vite de production réussi. Des avertissements de dépréciation Sass/Bootstrap et de directives use client de React Router restent présents.

**Dépendances :** npm audit indique zéro vulnérabilité connue au moment du contrôle du 21 octobre 2026. Ce résultat ne garantit pas l'absence de toute faille.

**Navigateur :** accueil, fiche et 404 vérifiés à 390, 768 et 1280 pixels avec une API simulée. Ouverture du menu, Échap, focus et fermeture à la navigation vérifiés sur mobile. Aucun débordement horizontal des accueils et aucune erreur JavaScript pageerror.

**Données :** schéma adapté et chargé en mémoire sous SQLite : quatre catégories, 15 spécialités, 14 villes, 17 artisans et trois artisans du mois. Contrôle des références et rejet d'une ville inexistante, d'une note invalide et d'une ville dupliquée.

**Prototype :** 189 interactions et trois points de départ enregistrés ; aucune destination invalide lors du contrôle structurel.

## Mise en service du 23 octobre 2026

Le site est accessible en HTTPS sur trouve-artisan.alwaysdata.net. Node.js 24 exécute Express et la base MariaDB 11.8 contient quatre catégories, 15 spécialités, 14 villes et 17 artisans. Le compte SQL a été limité à la lecture après l'import. Les secrets restent sur le serveur et ne sont pas joints aux fichiers publics.

Contrôles de l'API réelle : catégories et liste 200, fiche existante 200, artisan absent et route API inconnue 404, formulaire incomplet 400. La recherche et la fiche ont aussi été parcourues dans le navigateur. Les 15 tests Node passent sur l'hébergement, avec SQL simulé pour les tests automatisés.

Le service SMTP a été authentifié en TLS sur le port 465. Un message a été envoyé depuis le formulaire ; l'historique alwaysdata du 23 octobre 2026 à 16 h 48 indique « Envoyé » vers trouve-artisan@alwaysdata.net. La confirmation du site précise qu'il s'agit d'un message de démonstration. Le contenu de la boîte de réception n'a pas été ouvert pour vérifier sa lecture.

La version publique utilise 17 noms fictifs et des coordonnées de démonstration. CONTACT_TEST_TO dirige les messages vers la boîte du projet avec l'autorisation de son responsable ; aucun essai n'est envoyé à des artisans. Le jeu pédagogique initial reste disponible dans le livrable local, dans database/seed-pedagogique.sql.

Le build a été produit et vérifié localement puis transféré, la compilation sur l'offre d'hébergement ayant été interrompue par la limite de ressources. Les vérifications SQLite et les tests navigateur locaux décrits ci-dessus restent complémentaires aux contrôles MariaDB et SMTP réels. Un nouveau contrôle W3C HTML/CSS complet et une revue au lecteur d'écran restent à prévoir.

# 6. Veille de sécurité

## Méthode de suivi

La veille s’appuie sur npm audit, les avis GitHub Advisory Database et la lecture des versions présentes dans package-lock.json. Après une évolution de dépendance, les tests du formulaire, de l’API et le build doivent être relancés. Le contrôle npm du 21 octobre 2026 ne signalait aucune vulnérabilité connue ; il décrit l’état du registre à cette date.

## Avis étudiés

Nodemailer : l’avis GHSA-v53p-9fqp-m79j décrit un risque de déni de service dans l’analyse des adresses pour les versions jusqu’à 10.0.5. La version de référence publiée pour cet avis est 10.0.6. Le projet utilise la version 10.1.0, en dehors de cette plage ; le serveur impose aussi des limites de taille et de format aux champs reçus.

Avis GitHub Advisory Database - Nodemailer

uuid : l’avis GHSA-w5hq-g745-h8pq, CVE-2026-41907, concerne les contrôles de bornes des buffers de certaines fonctions. La branche 11 possède une version de référence 11.1.1. Le package.json impose uuid ^11.1.1 pour la dépendance de Sequelize par overrides. Ce choix doit rester suivi par les tests et la surveillance des avis.

Avis GitHub Advisory Database - uuid

## Portée

Un avis de sécurité signale un risque lié à une version et à un usage ; il ne prouve pas que le site a subi une attaque. L’absence d’alerte npm ne remplace pas la validation des entrées, la limitation des droits, la protection des secrets ni la surveillance de l’hébergement.

# 7. Maquettes et prototype

Consulter les maquettes Figma de référence

## Organisation des maquettes

Le fichier comporte 27 écrans : neuf pages pour chacun des formats ordinateur, tablette et mobile. Les en-têtes et pieds de page sont des instances de composants partagés. Les exports sont joints dans docs/maquettes.

## Parcours interactifs

Trois points de départ permettent de présenter les formats desktop, tablet et mobile. Depuis l'accueil, Rechercher ouvre le résultat Chocolaterie ; la carte correspondante mène à sa fiche. Le bouton Découvrir les artisans et la catégorie Bâtiment ouvrent la liste. Les liens du pied de page ouvrent les pages légales. Le logo et les liens de retour permettent de rejoindre l'accueil.

Dans Figma, sélectionner l'onglet Prototype, puis lancer Présenter. Les interactions sont enregistrées dans le fichier en ligne ; les images exportées restent statiques.

## Périmètre du prototype

La recherche est une simulation avec un résultat prévu. Les autres catégories et fiches ne sont pas représentées dans les 27 écrans. Le formulaire de la maquette n'envoie pas de courriel. Le menu mobile fonctionne dans le site ; les maquettes ne simulent pas son ouverture. Un essai manuel des trois parcours en mode Présenter complète le contrôle structurel.

## Ressources graphiques

Le logo et les illustrations sont placés dans public. La police Graphik Trial est fournie pour un usage personnel selon sa notice. Une licence web appropriée est nécessaire avant de publier ces fichiers ; Arial sert de police de secours.

# 8. Captures des maquettes

## Accueil sur ordinateur

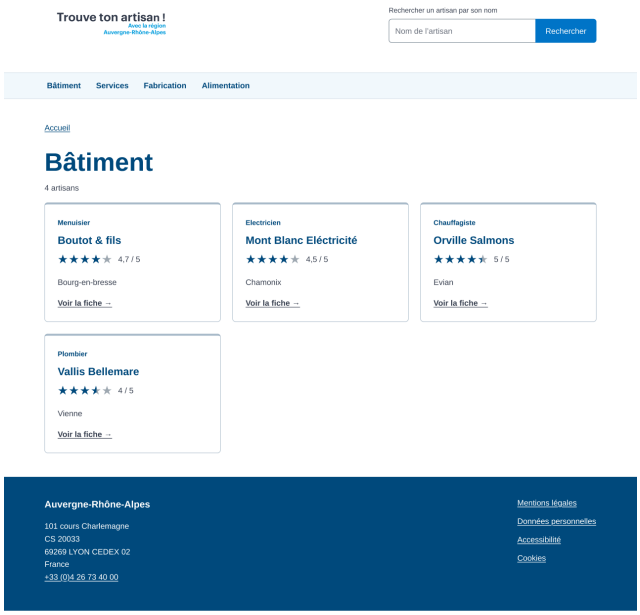
Export de la maquette Figma. Le prototype en ligne complète cette vue statique. Les noms du jeu de conception diffèrent des noms fictifs de la démonstration publique.

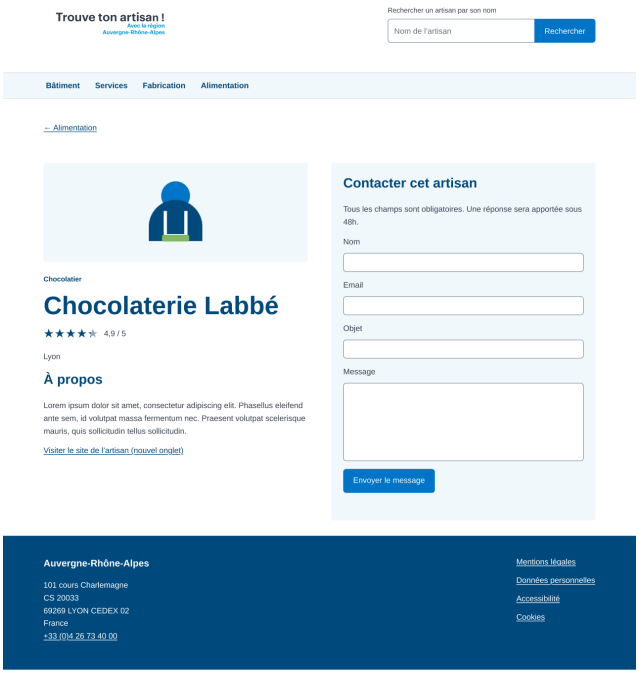


# 8. Captures des maquettes

## Liste et fiche artisan sur ordinateur

Exports Figma des deux étapes du parcours de consultation. Le fichier complet présente aussi les formats tablette et mobile.




Ouvrir les 27 écrans et leurs parcours dans Figma

# 9. Installation et mise en ligne

## Prérequis et configuration

Node.js 22.12 ou plus récent, npm, MySQL 8 ou MariaDB 10.6 ou plus récent, et un compte SMTP autorisé pour les messages. Dans le dossier trouve-ton-artisan, lancer npm ci. Copier .env.example vers .env et renseigner les variables DB et SMTP.

## Création de la base

Importer database/create.sql puis database/seed.sql avec un compte administrateur. **create.sql supprime la base trouve_ton_artisan existante avant de la recréer.** Sauvegarder les données à conserver avant son exécution. Créer ensuite un utilisateur SQL dédié avec le seul droit SELECT pour l'application.

Pour une base existante utilisant le champ artisans.city, sauvegarder, arrêter l'application et exécuter database/migrate-city.sql une seule fois. Tester d'abord cette opération sur une copie. Les DDL MySQL impliquent des commits implicites.

## Exécution

En développement : npm start pour le serveur sur le port 3001 et npm run dev pour Vite. Ouvrir http://localhost:5173 avec APP_ORIGIN configuré sur cette origine. Pour construire la version de production : npm test puis npm run build. npm start sert ensuite le dossier dist et l'API.

## Déploiement

Déployer ensemble les sources, le build dist et le schéma compatible. Définir NODE_ENV=production, APP_ORIGIN sur le domaine HTTPS, les accès SQL/SMTP et le nombre exact de proxys de confiance dans TRUST_PROXY. Conserver les secrets exclusivement sur l'hébergement.

Sur alwaysdata, la base est créée dans le panneau d'administration sous un nom préfixé. Les commandes DROP DATABASE, CREATE DATABASE et USE des scripts locaux ne doivent pas être exécutées telles quelles sur une base mutualisée : importer seulement le schéma et les données dans la base dédiée existante.

Après démarrage : ouvrir directement une fiche, effectuer une recherche, vérifier les erreurs, puis envoyer un message vers une boîte de test autorisée. Ne pas utiliser les adresses des artisans pour les essais. Vérifier l'acceptation SMTP et la réception effective.

## Suivi du projet

Le dépôt GitHub contient le code, les scripts SQL, les tests et le README. L'issue nº 1 et les demandes de fusion nº 2 et nº 3 matérialisent le suivi de la livraison et du formulaire. Les sources sont sur main ; les fichiers compilés destinés à l'hébergement sont sur deployment-build. Les mots de passe et les polices Trial sont exclus du dépôt public.

Consulter le dépôt GitHub

Consulter le site en ligne